

Roteiro: Revisão sobre Express.js

1. O que é o Express.js?

- Express.js é um framework minimalista para Node.js que facilita a criação de aplicações web e APIs.
- Ele simplifica o gerenciamento de rotas, middleware e requisições/respostas HTTP.
- Muito utilizado para construir servidores e aplicações RESTful.

2. Instalando o Express.js

2.1 Pré-requisitos

- Ter o [Node.js](#) instalado.
- Ter o gerenciador de pacotes npm (ou yarn).

2.2 Criando um projeto Node.js

npm init -y

npm install express

4. Criando e inicializando o servidor

4.1 Código básico para um servidor Express

Abra o arquivo index.mjs e adicione:

```
import express from 'express';

const app = express();

// Rota inicial
app.get('/', (req, res) => {
  res.send('Servidor Express rodando com ES Modules!');
});

// Inicializa o servidor
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

Configurando Views no Express

1. O que são Views no Express?

- **Views** são templates usados para gerar páginas HTML dinâmicas.
- O Express suporta vários motores de template, como **EJS, Pug e Handlebars**.

• 1. Estrutura do Projeto

- A estrutura inicial será assim:

meu-projeto-express/

```
| -- node_modules/
| -- package.json
| -- package-lock.json
| -- src/
|   |-- index.mjs
|   |-- views/
|     |-- index.html
|     |-- about.html
| -- public/
|   |-- css/
|     |-- style.css
|   |-- img/
|   |-- js/
```

❓ **views/** → Contém páginas HTML simples.

❓ **public/** → Arquivos estáticos (CSS, imagens, JavaScript).

Configuração inicial do express

```
// Instalar e importar o express
```

```

import express from 'express'
import path from 'path'

//Comando do path para acessar a pasta atual
const pastaAtual = process.cwd()
const pastaTemplate = path.join(pastaAtual, 'templates')
console.log(pastaTemplate)

// inicialização do aplicativo express
const app = express()
// Criação de uma porta do servidor
const porta = 5000

// Comando para transformar os dados vindos de formulários para objetos
javascript
app.use(express.urlencoded({extended:true}))
app.use(express.json())

// rota raiz
app.get('/', (req, res) => {
  res.sendFile(`${pastaTemplate}/index.html`)
})

app.listen(porta, ()=>{
  console.log(`Servidor iniciado na porta ${porta}`)
})

```

O que é o Nodemon?

O **Nodemon** é uma ferramenta que monitora mudanças nos arquivos do projeto e reinicia automaticamente o servidor Node.js sempre que detecta uma alteração. Isso evita que você precise parar e reiniciar o servidor manualmente sempre que fizer mudanças no código.

1. Instalando o Nodemon

Você pode instalar o **Nodemon** de duas formas:

1.1 Instalação Global (usável em qualquer projeto)

```
npm install -g nodemon
```

Após a instalação global, você pode rodar seu projeto com:

```
nodemon src/index.mjs
```

1.2 Instalação Local (somente para o projeto atual)

1.2 Instalação Local (somente para o projeto atual)

`npm install --save-dev nodemon`

Isso adiciona o **Nodemon** como uma dependência de desenvolvimento no seu projeto.

2. Configurando o Nodemon

2.1 Adicionando um script no package.json

Para facilitar a execução do Nodemon, edite o arquivo `package.json` e adicione o seguinte dentro de "scripts"

`"start": "nodemon index.mjs localhost 3000"`

Mostrar exemplos dos códigos sendo alterados em tempo real nas páginas html.

Explicação Completa sobre Rotas no Express

As **rotas** no Express são usadas para definir como o servidor responde a requisições HTTP (GET, POST, PUT, DELETE, etc.). Elas determinam **o que acontece quando um usuário acessa uma URL específica** do seu site ou aplicativo.

1. O que são rotas no Express?

Uma **rota** no Express é composta por três partes principais:

1. **O método HTTP** (GET, POST, PUT, DELETE, etc.).
2. **O caminho da URL** (ex: `/contato`, `/produtos`).
3. **Uma função callback** que é executada quando a rota é acessada.

Exemplo básico de uma rota GET no Express:

```
app.get('/sobre', (req, res) => {  
  res.send('Página Sobre');  
});
```

◆ Quando um usuário acessa **`http://localhost:5000/sobre`**, o servidor responde com 'Página Sobre'.

2. Tipos de Métodos HTTP

Os métodos HTTP definem **o que queremos fazer** com os dados. Os mais usados são:

Método	O que faz?	Exemplo de Uso
GET	Obtém dados do servidor	Buscar uma lista de produtos
POST	Envia dados para o servidor	Criar um novo usuário
PUT	Atualiza dados existentes	Atualizar o nome de um usuário
DELETE	Remove um dado do servidor	Excluir uma conta

- ◆ O Express permite definir rotas para cada um desses métodos!

📌 3. Criando Rotas no Express

3.1 Rota GET

O método **GET** é usado para **buscar informações do servidor**. Ele é acionado quando um usuário acessa uma página pelo navegador ou faz uma requisição HTTP GET.

```
app.get('/', (req, res) => {  
  res.send('Bem-vindo à página inicial!');  
});
```

- ◆ Acessando **http://localhost:5000/**, a mensagem "**Bem-vindo à página inicial!**" será exibida.

3.2 Rota GET com Parâmetros na URL

Podemos capturar valores diretamente na URL usando **parâmetros dinâmicos**.

```
app.get('/produto/:id', (req, res) => {  
  const produtoId = req.params.id;  
  res.send(`Produto de ID: ${produtoId}`);  
});
```

- ◆ Se acessarmos **http://localhost:5000/produto/42**, o servidor responderá com:

👉 "**Produto de ID: 42**"

- ◆ O req.params.id captura o valor passado na URL.

3.3 Rota GET com Query Params

Os **Query Params** são usados para enviar informações adicionais na URL, geralmente após o ?.

```
app.get('/buscar', (req, res) => {
  const nome = req.query.nome;
  const idade = req.query.idade;
  res.send(`Nome: ${nome}, Idade: ${idade}`);
});
```

◆ Se acessarmos **http://localhost:5000/buscar?nome=João&idade=25**, o servidor responderá com:

👉 **"Nome: João, Idade: 25"**

◆ O req.query captura os valores passados na URL como chave-valor.

3.4 Rota POST (Enviando Dados)

O método **POST** é usado quando queremos **enviar informações** ao servidor, geralmente via formulários.

Primeiro, precisamos ativar o **middleware** para interpretar os dados do formulário:

```
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

Agora, criamos uma rota **POST** para receber os dados:

```
app.post('/enviar', (req, res) => {
  const nome = req.body.nome;
  const email = req.body.email;
  res.send(`Dados recebidos: Nome - ${nome}, Email - ${email}`);
});
```

🔥 Como testar?

📄 Criamos um formulário HTML simples em views/form.html:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Formulário</title>
</head>
<body>
  <form action="/enviar" method="POST">
    <label>Nome:</label>
    <input type="text" name="nome">
    <br>
    <label>Email:</label>
```

```
    <input type="email" name="email">
    <br>
    <button type="submit">Enviar</button>
  </form>
</body>
</html>
```

❏ Criamos uma rota GET para exibir o formulário:

```
app.get('/form', (req, res) => {
  res.sendFile(path.join(__dirname, 'views', 'form.html'));
});
```

🔴 4. Outras Rotas Importantes

4.1 Rota PUT (Atualizar Dados)

O método **PUT** é usado para atualizar informações existentes no servidor.

```
app.put('/atualizar/:id', (req, res) => {
  const id = req.params.id;
  const novoNome = req.body.nome;
  res.send(`Usuário ${id} atualizado para ${novoNome}`);
});
```

- ◆ Para testar essa rota, você pode usar ferramentas como **Postman**.

4.2 Rota DELETE (Excluir Dados)

O método **DELETE** remove informações do servidor.

```
app.delete('/deletar/:id', (req, res) => {
  const id = req.params.id;
  res.send(`Usuário com ID ${id} foi removido`);
});
```

- ◆ Exemplo de chamada via **Postman**:

👉 DELETE <http://localhost:5000/deletar/42>

🔴 5. Lidando com Rotas Inexistentes

Se um usuário acessar uma rota que não existe, podemos criar um tratamento de erro:

```
app.use((req, res) => {
  res.status(404).send('Página não encontrada!');
});
```

```
});
```

Agora, se alguém tentar acessar uma URL inválida, verá essa mensagem.

◆ Questões usando GET

1️⃣ Questão Simples - Saudação Personalizada

Crie uma rota GET /saudacao que recebe um **nome** e uma **hora do dia** como **query params** (?nome=João&hora=18).

- Se a hora estiver entre **5 e 11**, a resposta deve ser "**Bom dia, João!**"
- Se a hora estiver entre **12 e 17**, a resposta deve ser "**Boa tarde, João!**"
- Se for **18 ou mais**, a resposta deve ser "**Boa noite, João!**"
- Caso a hora não seja informada, o sistema deve assumir que é **12h**.

✅ Exemplo de chamada:

GET /saudacao?nome=Maria&hora=9

💡 **Resposta:** "Bom dia, Maria!"

2️⃣ Questão Intermediária - Cálculo de Frete

Crie uma rota GET /frete que recebe três parâmetros na URL:

- peso (em kg)
- distancia (em km)
- tipo (normal ou expresso)

O cálculo do **frete** deve ser feito assim:

- **Se for normal:** $\text{peso} * 2 + \text{distancia} * 0.5$
- **Se for expresso:** $\text{peso} * 3 + \text{distancia} * 0.8$

A resposta deve mostrar o **valor do frete** formatado com **duas casas decimais**.

✅ Exemplo de chamada:

GET /frete?peso=10&distancia=200&tipo=expresso

💡 **Resposta:** "O valor do frete é R\$260.00"

◆ Questões usando POST

3️⃣ Questão Simples - Cadastro de Usuário

Crie uma rota POST /cadastro que recebe um **nome**, **email** e **idade** de um usuário pelo **corpo da requisição**.

- O servidor deve retornar uma mensagem de sucesso contendo o nome e a idade.
- Se a idade for menor que **18**, deve exibir um aviso: **"Cadastro permitido apenas para maiores de idade!"**

✅ Exemplo de envio (formulário ou JSON):

```
{
  "nome": "Carlos",
  "email": "carlos@email.com",
  "idade": 17
}
```

💡 Resposta: "Cadastro permitido apenas para maiores de idade!"

❏ Questão Avançada - Cálculo de Média e Classificação

Crie uma rota POST /avaliacao que recebe pelo formulário três notas de um aluno (nota1, nota2, nota3).

- A média é calculada como $(\text{nota1} + \text{nota2} + \text{nota3}) / 3$.
- A resposta deve classificar o aluno conforme a média:
 - **Média ≥ 7 :** "Aprovado!"
 - **Média entre 5 e 6.9:** "Recuperação!"
 - **Média < 5 :** "Reprovado!"

✅ Exemplo de envio (formulário ou JSON):

```
{
  "nota1": 8,
  "nota2": 6,
  "nota3": 9
}
```

💡 Resposta: "Aprovado!"

🔥 Guia Completo sobre Handlebars no Express.js 🚀

Agora que já entendemos o funcionamento das **rotas GET e POST**, é hora de deixar nossas páginas **dinâmicas**! Para isso, vamos usar um **Template Engine**, e a nossa escolha será o **Handlebars** (hbs).

📌 1. O que é o Handlebars?

O **Handlebars** é um **motor de templates** (Template Engine) que permite: ✅ Criar **páginas dinâmicas** sem precisar escrever HTML repetitivo.

- ✅ Utilizar **variáveis** dentro do HTML.
- ✅ Criar **layouts reutilizáveis** para evitar duplicação de código.
- ✅ **Condicionais (if e else)** para mostrar ou ocultar partes da página.
- ✅ **Laços (each)** para percorrer e exibir listas.

Ele funciona processando arquivos **.hbs** e preenchendo com os **dados enviados pelo Node.js**.

📌 2. Instalando o Handlebars no Express

Para utilizar o **Handlebars** no Express, instale o pacote **express-handlebars**:

```
npm install express-handlebars
```

📌 3. Estruturando o Projeto

Vamos organizar as pastas do nosso projeto para usar o Handlebars corretamente.

```
/meu-projeto
├── /views
│   ├── /layouts
│   │   └── main.hbs      # Layout base (cabeçalho e rodapé)
│   ├── index.hbs        # Página inicial
│   └── contato.hbs      # Outra página
├── /public               # Arquivos estáticos (CSS, imagens)
└── index.mjs             # Arquivo principal
```

📌 4. Configurando o Handlebars no Express

No arquivo **index.mjs**, configuramos o Express para usar o Handlebars como engine de templates:

```
import express from 'express'
import { engine } from 'express-handlebars'
import path from 'path'

const app = express()
```

```

const porta = 5000

// Configurando o Handlebars como Template Engine
app.engine('hbs', engine({ extname: '.hbs' }))
app.set('view engine', 'hbs')
app.set('views', path.join(process.cwd(), 'views'))

// Utilização de formulários
app.use(express.urlencoded({ extended: true })) // Permite receber dados
de formulários
app.use(express.json()) // Permite receber dados no formato JSON

// Servir arquivos estáticos da pasta "public"
app.use(express.static(path.join(process.cwd(), 'public')))

// Rota para a página inicial
app.get('/', (req, res) => {
  res.render('index', { titulo: 'Página Inicial', mensagem: 'Bem-vindo
ao site!' })
})

// Inicializa o servidor
app.listen(porta, () => {
  console.log(`Servidor rodando na porta ${porta}`)
})

```

🔥 5. Criando a Página Principal (index.hbs)

Agora, dentro da pasta views, criamos o arquivo index.hbs:

```

<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>{{titulo}}</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>{{mensagem}}</h1>
</body>
</html>

```

✅ Aqui estamos usando `{{titulo}}` e `{{mensagem}}`, que vêm do `res.render()`.

🔥 6. Criando um Layout Base (main.hbs)

Para evitar repetir HTML em todas as páginas, criamos um **layout base** no arquivo `views/layouts/main.hbs`:

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>{{titulo}}</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <header>
    <h1>Meu Site</h1>
    <nav>
      <a href="/">Início</a>
      <a href="/contato">Contato</a>
    </nav>
  </header>

  {{{body}}} <!-- Aqui será inserido o conteúdo das páginas -->

  <footer>
    <p>© 2025 Meu Site</p>
  </footer>
</body>
</html>
```

Agora, podemos modificar `index.hbs` para usar esse layout:

```
{{!-- views/index.hbs --}}
{{#extends "main"}}

{{#block "body"}}
<h2>{{mensagem}}</h2>
{{/block}}
```

✓ Com isso, todas as páginas herdam a estrutura do `main.hbs`.

🔥 7. Usando IF e ELSE no Handlebars

Podemos exibir conteúdos **condicionalmente** usando `{{#if}}` e `{{else}}`.

Exemplo: Mostrando um aviso se o usuário estiver logado ou não.

```
{{#if logado}}
  <p>Bem-vindo, {{nome}}!</p>
{{else}}
  <p>Você precisa fazer login.</p>
```

```
{{/if}}
```

No index.mjs, passamos a variável:

```
app.get('/', (req, res) => {  
  res.render('index', { logado: false, nome: 'Carlos' })  
})
```

💡 **Dica:** O Handlebars **não suporta operadores como &&, ||, >= diretamente**, então se precisar de lógica mais avançada, passe os valores prontos do backend.

📌 8. Usando EACH para Listas

Para renderizar **listas**, usamos `{{#each}}`.

Exemplo: Exibindo produtos em uma loja:

```
<h2>Produtos disponíveis</h2>  
<ul>  
  {{#each produtos}}  
    <li>{{nome}} - R$ {{preco}}</li>  
  {{/each}}  
</ul>
```

No index.mjs, enviamos a lista:

```
app.get('/', (req, res) => {  
  const produtos = [  
    { nome: 'Notebook', preco: 2500 },  
    { nome: 'Mouse', preco: 120 },  
    { nome: 'Teclado', preco: 200 }  
  ]  
  res.render('index', { produtos })  
})
```

📌 9. O que NÃO dá para fazer no Handlebars?

Apesar de ser uma ferramenta poderosa, o Handlebars tem **limitações**: ❌ **Não suporta expressões JavaScript dentro do template** (ex.: `{{numero + 1}}` não funciona).

- ✗ **Não permite criar funções complexas dentro do HTML.**
- ✗ **Não possui suporte nativo para formatação de datas e números** (é necessário usar **helpers personalizados**).

💡 **Dica:** Sempre que precisar de lógica mais avançada, **prepare os dados antes** no backend e envie prontos para o template.

🔥 10. Boas Práticas no Handlebars

- ✓ **Separe as responsabilidades: HTML no Handlebars e lógica no Node.js.**
- ✓ **Use layouts (main.hbs)** para não repetir código.
- ✓ **Prefira Helpers** para lógica mais avançada.
- ✓ **Mantenha as views organizadas** com subpastas se necessário (views/produtos/lista.hbs).
- ✓ **Utilize arquivos estáticos corretamente** (/public para CSS, JS e imagens).

📄 **Questão: Sistema de Cadastro e Listagem de Produtos**

Você foi contratado para desenvolver um sistema básico de gerenciamento de produtos para um pequeno comércio. O sistema deve permitir o cadastro de produtos via formulário, armazenar os produtos temporariamente em memória e listá-los em uma página.

🖨️ **Requisitos do Sistema:**

🏠 **Página inicial (/)**

- Deve conter um menu com links para "Cadastrar Produto" e "Lista de Produtos".
- Deve ter uma mensagem de boas-vindas e uma breve explicação sobre o sistema.

📝 **Página de Cadastro (/cadastro)**

- Deve conter um formulário com os seguintes campos:
 - Nome do Produto (campo obrigatório)
 - Preço (campo obrigatório)
 - Quantidade em estoque (campo obrigatório)
- Deve enviar os dados via POST para a rota /salvar-produto.

📡 **Rota /salvar-produto (POST)**

- Deve receber os dados do formulário e armazená-los em uma lista de produtos.
- Após salvar, redirecionar para a página de listagem.

📄Página de Listagem (/produtos)

- Deve exibir todos os produtos cadastrados.
- Se não houver produtos cadastrados, mostrar a mensagem "**Nenhum produto cadastrado ainda.**"
- Para cada produto, exibir:
 - Nome
 - Preço formatado em reais (R\$)
 - Quantidade em estoque
- Caso um produto tenha quantidade **menor ou igual a 5**, exibir a mensagem em vermelho "**Estoque baixo!**".

🔧 Dicas Técnicas:

- Utilize **layouts** para reutilizar o HTML padrão.
- Use `each` para listar os produtos cadastrados.
- Utilize `if/else` para exibir mensagens quando o estoque estiver baixo.
- Após a submissão do formulário, os produtos devem ser armazenados temporariamente em uma variável (não usar banco de dados ainda).

🏆 O que será avaliado?

- ✅ Implementação correta das rotas (GET e POST).
- ✅ Uso adequado de **Handlebars** para exibir os dados dinamicamente.
- ✅ Uso de `if/else` para verificar o estoque baixo.
- ✅ Uso de `each` para listar os produtos.
- ✅ Organização do código, estrutura das pastas e reaproveitamento de layouts.